

Система обмена сообщениями с криптографической защитой информации

Зюков Александр

КЭП-01-24

Ховроненко Андрей

КЭП-03-24

LiteBroker

Система обмена сообщениями с криптографической защитой информации

Kafka broker

AES-256-GCM
crypto containers

Sensor topic
random telemetry

Grafana
observability

```
{ algorithm, iv, ciphertext, authTag, metadata }
```

Что представляет собой продукт

LiteBroker — локальный учебный мессенджер с выбором собеседника.

Каждый диалог публикуется в отдельный Kafka topic: `litebroker.chat.<userA>__<userB>`.

Замеры датчиков генерируются автоматически и публикуются в topic `litebroker.sensors.random`.

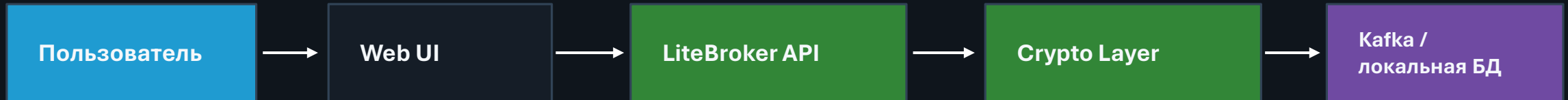
Сообщения сохраняются в локальная БД и Kafka не открытым текстом, а криптоконтейнером.

Prometheus, Loki/Promtail и Grafana используются для наблюдаемости и демонстрации аудита.

Ценность продукта:
конфиденциальность сообщения,
целостность контейнера,
разделение потоков данных

Неприемлемое событие:
сообщение или служебный поток обходит
политику и появляется в системе как
открытый текст

Концепция безопасности продукта



Сценарий 1: пользователь входит, выбирает собеседника, отправляет сообщение.

Сценарий 2: генератор датчиков создает телеметрию без участия пользователя.

Сценарий 3: администратор смотрит состояние системы в Grafana/Kafka UI.

Граница доверия: браузер не работает напрямую с Kafka и БД.

Цели и предположения безопасности

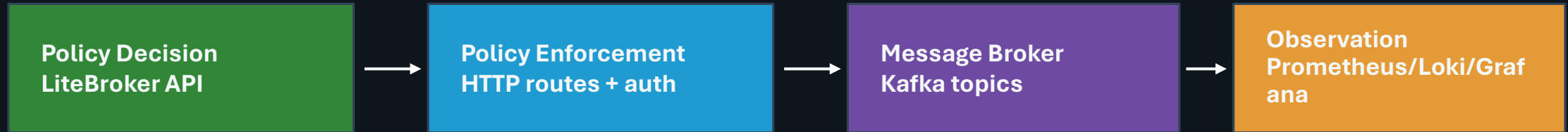
Цели безопасности

не передавать сообщение как открытый текст;
сохранять целостность через GCM authTag;
разделить чат и датчики по разным topics;
не дать frontend обойти API и политику;
дать наблюдаемость без доступа к управлению сообщениями.

Предположения безопасности

проект учебный и запускается локально;
секрет шифрования задается через env;
Kafka в демо без TLS/SASL, но архитектура допускает усиление;
локальная БД — локальное хранилище приложения;
внешние реальные аккаунты и Telegram API не используются.

Архитектура политики

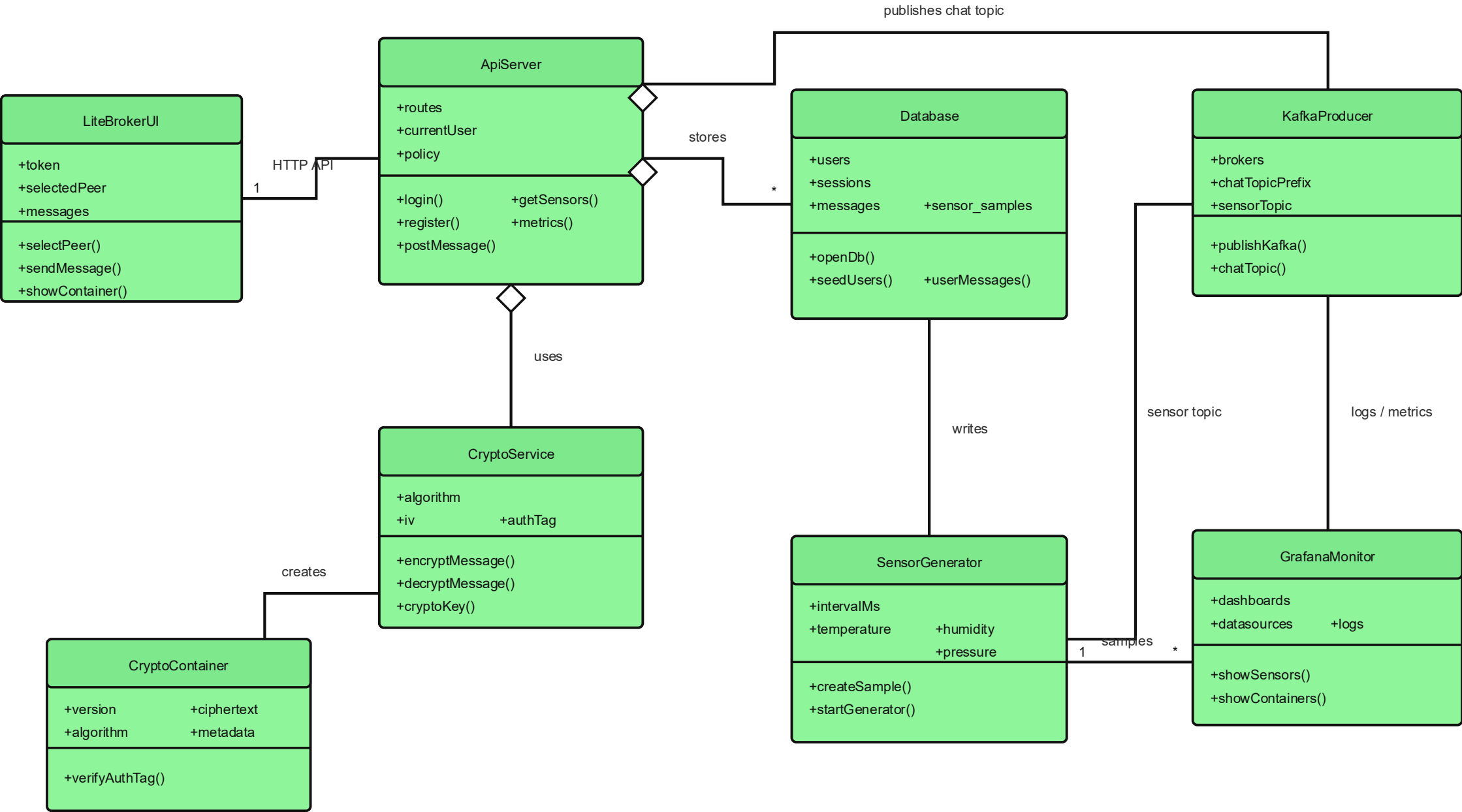


Разрешены только операции через API: login/register/users/messages/sensors/metrics.

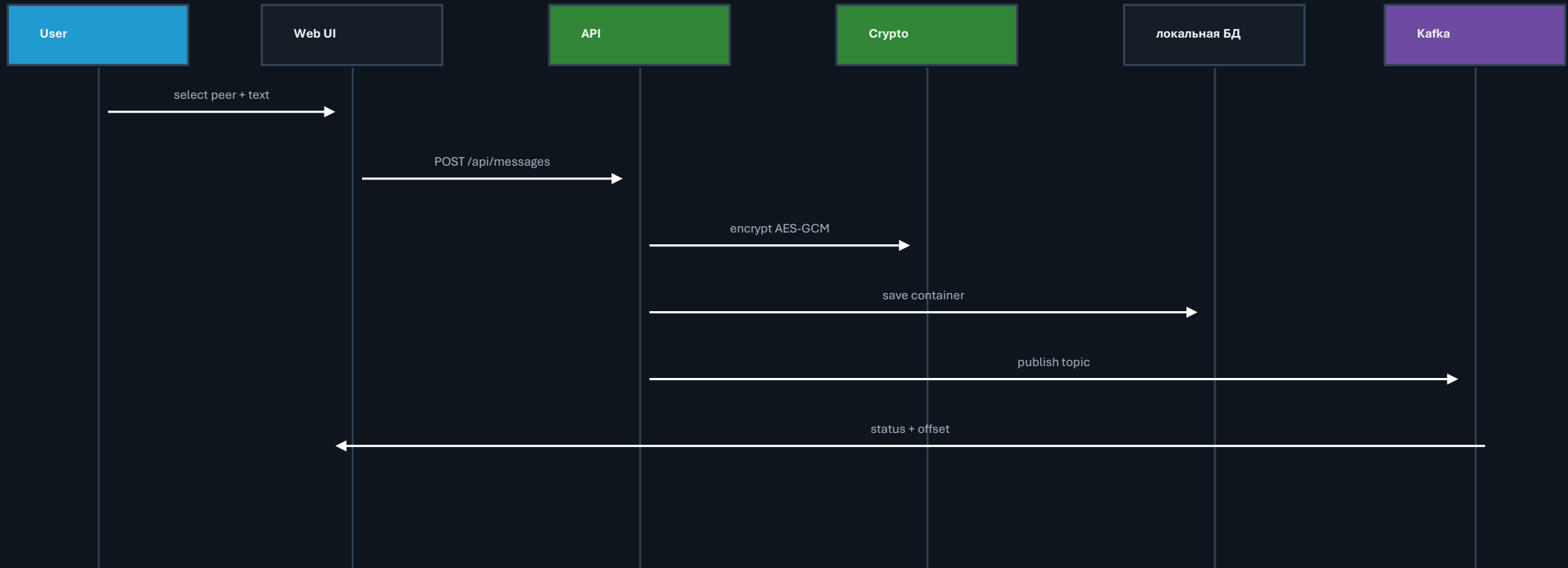
Для сообщений обязательны: авторизованный пользователь, выбранный recipient, crypto container.

Для каждого домена задается разрешенный тип взаимодействия: chat topic или sensor topic.

Все, что не описано в политике, считается неразрешенным по умолчанию.



Базовый сценарий отправки сообщения



Механизмы изоляции и контроля

Docker Compose: отдельные контейнеры для LiteBroker, Kafka, Kafka UI, Prometheus, Loki/Promtail, Grafana.

Kafka topics: отдельный поток для каждого чата и отдельный поток для датчиков.

API boundary: браузер не получает доступ к локальной БД/Kafka напрямую.

Crypto boundary: весь текст проходит через AES-256-GCM до сохранения/публикации.

Monitoring boundary: Grafana наблюдает метрики и логи, но не управляет сообщениями.

**Почему так:
простые границы доверия,
меньше связность,
понятная демонстрация для
защиты,
легко проверить в Docker/Kafka
UI/Grafana.**

Политики безопасности и шаблоны security-by-design

Раздельное принятие и применение решений о безопасности: API проверяет пользователя и допустимость операции, Kafka только переносит разрешенное событие.

Выделенный обработчик для очистки данных: входные JSON-запросы нормализуются и валидируются на backend.

Выделенный механизм поддержания состояния безопасности: /api/health, /metrics, Prometheus/Grafana показывают состояние Kafka и приложения.

Минимизация поверхности атаки: наружу вынесены только нужные порты и HTTP API.

Наблюдаемость и аудит: JSONL-логи контейнеров доступны через Loki/Grafana.

Паттерны проектирования ПО

Facade

UI работает через REST API и не знает деталей
Kafka/локальная БД.

Strategy

Криптографический слой можно заменить другим
алгоритмом.

Mediator

Kafka выступает посредником между доменами и снижает
связность.

Observer

Prometheus, Loki и Kafka UI наблюдают события и
состояние.

Template Method

Единый pipeline сообщения: validate → encrypt → save →
publish.

Factory Method

Создание crypto container как отдельная фабрика
структуры события.

Как реализовано шифрование

```
{
  "version": 2,
  "algorithm": "AES-256-GCM",
  "iv": "...",
  "ciphertext": "...",
  "authTag": "...",
  "metadata": {
    "messageId": "...",
    "senderId": "...",
    "recipientId": "...",
    "chatTopic": "litebroker.chat.alice__bob",
    "createdAt": "..."
  }
}
```

`ciphertext` скрывает текст сообщения.

`iv` делает шифрование разных сообщений уникальным.

`authTag` проверяет целостность контейнера.

`metadata` связывает сообщение с пользователем, получателем и Kafka topic.

Пароли не шифруются, а хешируются через PBKDF2 + salt.

Наблюдаемость: 2 дашборда

Dashboard 1

Показания датчиков

Prometheus metrics:

temperature / humidity / pressure / total

Dashboard 2

Логи чата

Loki logs:

chat-containers.jsonl + счетчики topic

Первый dashboard подтверждает, что sensor topic живой.
Второй dashboard показывает криптоконтейнеры и аудит сообщений.
Promtail читает только папку логов LiteBroker.

Подход к тестированию проектного кода

Статическая проверка: ``node --check LiteBroker/server.mjs`` и ``node --check LiteBroker/public/app.js``.

Проверка инфраструктуры: ``docker compose config --quiet``.

Функциональный тест: регистрация, выбор собеседника, отправка сообщения.

Kafka-тест: в Kafka UI появляются chat topic и sensor topic.

Security-тест: в логах и Kafka виден JSON crypto container, а не открытый текст.

Observability-тест: Grafana показывает датчики и логи криптоконтейнеров.

Негативные сценарии: неверный пароль, пустое сообщение, отсутствие получателя, недоступность Kafka.

Соответствие идее монитора безопасности

Non-bypassable
все операции идут через LiteBroker API

Evaluatable
компоненты маленькие: UI, API, crypto, DB, Kafka, monitoring

Always-invoked
каждая отправка проходит
validate/encrypt/save/publish

Tamperproof
конфигурация вынесена в Docker/env;
мониторинг read-only

Почему выбраны именно эти решения

Kafka выбрана как брокер сообщений: она естественно показывает изолированные домены и политику взаимодействия через topics.

AES-256-GCM выбран потому, что дает конфиденциальность и контроль целостности одним контейнером.

локальная БД выбран для локального учебного хранения без внешней БД.

Docker Compose выбран для воспроизводимого запуска и изоляции сервисов.

Grafana/Prometheus/Loki выбраны для защиты проекта: видно состояние, датчики и аудит криптоконтейнеров.

UML используется для объяснения структуры и сценария.